

Use Cases

If your application frequently queries a field that contains an array value, a multikey index improves performance for those queries.

Indexing commonly queried fields increases the chances of **covering** those queries. Covered queries are queries that can be satisfied entirely using an index, without examining any documents. This optimizes query performance.

For example, documents in a `students` collection contain a `test_scores` field: an array of test scores a student received throughout the semester. You regularly update a list of top students: students who have at least five `test_scores` greater than 90.

You can create an index on the `test_scores` field to improve performance for this query. Because `test_scores` contains an array value, MongoDB stores the index as a multikey index.

Get Started

To create a multikey index, see:

- Create an Index on an Array Field
- Create an Index on an Embedded Field in an Array

Details

This section describes technical details and limitations for multikey indexes.

Index Bounds

The bounds of an index scan define the parts of an index to search during a query. The computation of multikey index bounds follows special rules. For details, see [Multikey Index Bounds](#).

Unique Multikey Indexes

In a **unique** multikey index, a document may have array elements that result in repeating index key values as long as the index key values for that document do not duplicate those of another document.

To learn more and see an example of this behavior, see [Unique Constraint Across Separate Documents](#).

Compound Multikey Indexes

In a **compound** multikey index, each indexed document can have *at most* one indexed field whose value is an array. Specifically:

- You cannot create a compound multikey index if more than one field in the index specification is an array. For example, consider a collection that contains this document:

```
{ _id: 1, scores_spring: [ 8, 6 ], scores_fall: [ 5, 9 ] }
```

You can't create the compound multikey index `{ scores_spring: 1, scores_fall: 1 }` because both fields in the index are arrays.

- If a compound multikey index already exists, you cannot insert a document that would violate this restriction.

Consider a collection that contains these documents:

```
{ _id: 1, scores_spring: [8, 6], scores_fall: 9 }  
{ _id: 2, scores_spring: 6, scores_fall: [5, 7] }
```

You can create a compound multikey index `{ scores_spring: 1, scores_fall: 1 }` because for each document, only one field indexed by the compound multikey index is an array. No document contains array values for both `scores_spring` and `scores_fall` fields.

However, after you create the compound multikey index, if you attempt to insert a document where both `scores_spring` and `scores_fall` fields are arrays, the insert fails.

Sorting

When you sort based on an array field that is indexed with a **multikey index**, the query plan includes a **blocking sort** stage unless both of the following are true:

- The index **boundaries** for all sort fields are `[MinKey, MaxKey]`.
- No boundaries for any multikey-indexed field have the same path prefix as the sort pattern.

Shard Keys

You cannot specify a multikey index as a shard key index.

However, if the shard key index is a **prefix** of a compound index, the compound index may become a compound *multikey* index if one of the trailing keys (that are not part of the shard key) indexes an array.

Hashed Indexes

Hashed indexes cannot be multikey.

Covered Queries

Multikey indexes cannot **cover** queries over array fields. However, multikey indexes can cover queries over non-array fields if the index tracks which field or fields cause the index to be multikey.

For example, consider a `matches` collection with these documents:

```
db.matches.insertMany( [  
  { name: "joe", event: ["open", "tournament"] },  
  { name: "bill", event: ["match", "championship"] }  
)
```

The `matches` collection has a compound multikey index on the `name` and `event` fields:

```
db.matches.createIndex( { name: 1, event: 1 } )
```

This index is a multikey index because the `event` field contains array values.

The multikey index covers the following query, even though the matched field (`name`) is not an array:

```
db.matches.find( { name: "bill" } )
```

Because `name` field is part of the index **prefix**, the index covers queries on the `name` field. The index cannot cover queries on both `name` and `event`, because multikey indexes cannot cover queries on array fields.

Query on an Array Field as a Whole

When a query filter specifies an **exact match for an array as a whole**, MongoDB can use the multikey index to look up the *first* element of the query array, but cannot use the multikey index scan to find the whole array.

Instead, after using the multikey index to look up the first element of the query array, MongoDB retrieves the associated documents and filters for documents whose array matches the array in the query.

For example, consider an `inventory` collection that contains these documents:

```
db.inventory.insertMany( [  
  { _id: 5, type: "food", item: "apple", ratings: [ 5, 8, 9 ] },  
  { _id: 6, type: "food", item: "banana", ratings: [ 5, 9 ] },  
  { _id: 7, type: "food", item: "chocolate", ratings: [ 9, 5, 8 ] },  
  { _id: 8, type: "food", item: "fish", ratings: [ 9, 5 ] },  
  { _id: 9, type: "food", item: "grapes", ratings: [ 5, 9, 5 ] }  
)
```

The `inventory` collection has a multikey index on the `ratings` field:

```
db.inventory.createIndex( { ratings: 1 } )
```

The following query looks for documents where the `ratings` field is the array `[ 5, 9 ]`:

```
db.inventory.find( { ratings: [ 5, 9 ] } )
```

MongoDB can use the multikey index to find documents that have 5 at any position in the `ratings` array. Then, MongoDB retrieves these documents and filters for documents whose `ratings` array equals the query array `[ 5, 9 ]`.

\$expr

The **\$expr** operator does not support multikey indexes.

Learn More

- To learn how MongoDB combines multikey index bounds to improve performance, see [Multikey Index Bounds](#).
- To learn how to query array fields, see:
 - [Query an Array](#)
 - [Query an Array of Embedded Documents](#)